

Interconnection Structures

Condensed Study Notes

Generated: 2026-04-28 22:42 UTC

COMPUTER ARCHITECTURE & ORGANIZATION LECTURE (WEEK 2)

This lecture focuses on the fundamental components of a computer system and how they interact, building upon the concept of the CPU's role.

Key components discussed:

- **Central Processing Unit (CPU):** The brain of the computer, responsible for executing instructions. It operates using the fetch-decode-execute cycle.
- **Fetch:** Retrieves an instruction from memory.
- **Decode:** Interprets the instruction.
- **Execute:** Carries out the instruction.
- **Memory:** Stores data and instructions for the CPU. It's like a temporary workspace for the CPU.
- **Input/Output (I/O) Devices:** Allow the computer to interact with the outside world (e.g., keyboard, screen, network card). These are the computer's senses and voice.
- **System Bus:** A communication pathway connecting the CPU, memory, and I/O devices. Think of it as the highway system within the computer.

Interconnection Structures

Computers consist of three main modules: the Processor, Memory, and I/O devices.

These modules need paths to communicate with each other.

The collection of these connecting paths is called the **interconnection structure**. Its design is determined by the type of data exchanges required between the modules.

I/O module

From an internal perspective, an I/O module functions similarly to memory. An I/O module can manage multiple external devices, each accessible via a unique address called a port.

Interconnection structures must support several types of data transfers:

- **Memory to processor:** The processor retrieves instructions or data from memory.
- **I/O to processor:** The processor reads data from an external device through the I/O module.
- **Processor to I/O:** The processor sends data to an external device.
- **Processor to memory:** The processor writes data to memory.
- **I/O to or from memory:** The I/O module can directly exchange data with memory without involving the processor, a process known as Direct Memory Access (DMA).

In addition to data transfers, an I/O module can send interrupt signals to the processor to signal events, and it uses control signals to manage system operations.

Memory

Memory stores data and instructions. Each location in memory is identified by a unique numerical address, starting from 0 up to N-1.

A typical memory module contains N words, and each word has the same length.

Operations on memory are controlled by read and write signals, which determine whether data is retrieved from or stored into a specific address.

Complex Instruction Set Architecture (CISC)

CISC aims to increase CPU performance by making the hardware more complex. This complexity allows a single instruction to perform a sequence of low-level operations, such as loading data, performing an operation, and storing the result. For example, a multiplication command in CISC might handle the loading of operands, the calculation, and the storing of the product all within one instruction.

Instruction set Architecture: CISC, RISC

Instruction Set Architecture (ISA) defines the set of commands a CPU understands.

CISC (Complex Instruction Set Computing) architectures use complex hardware to execute single instructions that can perform multiple operations. This aims to boost CPU performance by reducing the number of instructions needed for a task.

RISC (Reduced Instruction Set Computing) is an alternative approach.

Reduced Instruction Set Architecture (RISC)

RISC focuses on simplifying hardware by using a small set of basic instructions. These instructions handle fundamental operations like loading data, performing evaluations, and storing results. Each instruction performs a single, simple step, unlike the multi-operation instructions found in CISC architectures.

Key idea: Simpler hardware through a minimal, basic instruction set.

The “best” way to design a CPU has been a subject of debate

The design of a CPU, specifically regarding its instruction set, has been a long-standing debate. This debate centres on whether CPUs should utilize a large set of complex instructions that can perform multiple operations in a single step (CISC), or a smaller, simpler set of instructions requiring more steps for complex tasks but potentially executing each step faster (RISC).

- **CISC (Complex Instruction Set Computer):** Employs a large collection of complex machine language instructions.
- **RISC (Reduced Instruction Set Computer):** Uses a smaller, more basic set of simple instructions.

Characteristic of RISC

RISC (Reduced Instruction Set Computer) architectures are characterized by a set of design principles that simplify the CPU's operation.

Key Characteristics:

- **Simpler Instructions:** Instructions are basic and easy to decode, streamlining the CPU's interpretation process.
- **Single-Word Instructions:** Each instruction typically fits within a single machine word, making them uniform in size.
- **Single-Cycle Execution:** Most instructions are designed to complete their execution within a single clock cycle.
- **Simple Addressing Modes:** The ways data can be accessed are straightforward and limited.
- **More General-Purpose Registers:** A larger number of registers are available for storing data, reducing the need to access main memory frequently.
- **Fewer Data Types:** The architecture supports a reduced set of data types, contributing to overall simplicity.
- **Pipeline Achievable:** The simple and uniform nature of RISC instructions makes them well-suited for instruction pipelining, a technique that allows multiple instructions to be processed concurrently.

Characteristic of CISC

CISC (Complex Instruction Set Computer) architectures are characterized by:

1. **Complex Instructions:** These instructions are intricate and require complex decoding.
2. **Larger Instruction Size:** Instructions can exceed a single word in size.
3. **Multi-Cycle Execution:** Instructions may take more than one clock cycle to complete.
4. **Fewer Registers:** CISC relies less on general-purpose registers, performing operations directly in memory.
5. **Complex Addressing Modes:** CISC utilizes sophisticated ways to access data in memory.

In contrast, the RISC approach breaks down operations like addition into separate load, operate, and store steps. While RISC programs are longer and need more memory, they require fewer transistors due to simpler commands.

CISC vs. RISC: Which is Better?

The debate on whether CISC or RISC is superior has no definitive answer, as both have advantages and disadvantages.

- **CISC:** Aims to reduce the number of instructions per program, but this often means more cycles are needed per instruction.
- **RISC:** Aims to reduce the cycles per instruction, but this typically requires more instructions per program.

Historically, CISC emerged when assembly programming was common and tedious, requiring complex instructions to simplify coding. With the rise of high-level languages, the reliance on assembly decreased, and RISC architectures gained prominence.

Examples:

- **CISC:** x86 CPUs (common in desktops/laptops)
- **RISC:** ARM (common in mobile devices), PowerPC, SPARC

Difference

Here's a breakdown of the key differences between CISC and RISC architectures:

- **Focus:** CISC focuses on **hardware** complexity to execute instructions, while RISC focuses on **software** simplicity.
- **Registers:** RISC requires a larger number of registers, whereas CISC requires fewer.
- **Operations:** CISC can perform operations involving memory (e.g., Memory to Memory), while RISC is limited to Register to Register operations.
- **Instruction Size:** RISC uses fixed-sized instructions, typically fitting within one word. CISC uses variable-sized instructions.
- **Transistor Usage:** In RISC, transistors are primarily used for more registers. In CISC, transistors are used for storing complex instructions.
- **Execution Time:** RISC instructions generally execute in a single clock cycle. CISC instructions often take more than one clock cycle.
- **Code Size:** RISC architectures tend to have a smaller code size due to simpler instructions. CISC architectures can result in a larger code size.

Instruction and Instruction Types

Beyond the basic RISC/CISC distinction, computer instruction sets can be classified by several characteristics. The instruction set acts as the interface between software and hardware, defining the actions the hardware will perform.

Key characteristics of instructions include:

- **Addressing modes:** How the instruction specifies the location of data or operands.
- **Numbers of operands:** The quantity of data items an instruction operates on.
- **Types of operations supported:** The range of actions the instruction can perform (e.g., arithmetic, logic, data transfer).
- **Fixed versus variable length:** Whether all instructions occupy the same memory space or vary in size.

Word length

Processors can be characterized by their word length (e.g., 4-bit, 8-bit, 16-bit, 32-bit). In some architectures, the length of a data word, an instruction, and an address are the same.

However, especially in computers designed for smaller data words, instructions and addresses might be longer than the basic data word.

Processors can also be classified by instruction execution:

- **Single-issue processors:** Execute one instruction at a time, even if multiple instructions are in different stages of processing.
- **Multiple-issue processors (e.g., superscalar):** Use logic to identify and execute instructions simultaneously.

The set of registers accessible to programs is called the programming model (or programmer model). The CPU has additional internal registers not available to programmers.

INSTRUCTION SEQUENCING

Instructions are executed through a two-phase procedure:

1. **Instruction Fetch:** The instruction at the memory address stored in the Program Counter (PC) is retrieved and loaded into the Instruction Register (IR) within the processor.
2. **Instruction Execute:** The instruction in the IR is examined to determine the operation to perform.

The processor's control circuits use the PC to fetch and execute instructions sequentially, typically in order of increasing memory address. This is known as **straight-line sequencing**.

There are four fundamental types of operations:

1. **Data Transfer:** Moving data between memory and processor registers.
2. **Arithmetic & Logic Operations:** Performing calculations and logical comparisons on data.
3. **Program Sequencing & Control:** Directing the flow of program execution.
4. **I/O Transfers:** Communicating with input/output devices.

Instructions can be classified by their addressing scheme:

- **Zero address instructions:** Operands are implicitly managed, often using a structure like a push-down stack.
- **One address instructions:** Use a single explicit operand; the other is implicitly the accumulator (e.g., Add contents of A to accumulator & store sum back to accumulator).
- **Two address instructions:** Use two explicit operands (e.g., $B \leftarrow [A] + [B]$).
- **Three address instructions:** Use three explicit operands (e.g., Adding contents of R1, R2 & place sum in R3).

Register Transfer Notation (RTN) is used to represent these operations, where the left-hand side names a location and the right-hand side denotes a value.